

Document 11: Product Packaging, Feature Flags & Natural Intellects Control Center Architecture

Project: Natural Intellects School Management System (NISMS)

Version: 1.0

Status: Draft

Purpose:

This document defines how the Natural Intellects platform manages subscriptions, feature access, upgrades, pricing tiers, and centralized control of multiple school systems through a unified administrative dashboard.

It introduces:

- Feature flag architecture
- Subscription packaging
- Central control dashboard
- Upgrade mechanics
- Licensing model
- Runtime feature control system

This document is critical for:

- SaaS evolution
- Revenue model implementation
- System scalability
- Upgrade automation
- Multi-school management

TABLE OF CONTENTS

1. Product Strategy Overview
2. Platform vs School Systems
3. Feature Flag Architecture
4. Subscription Plan System
5. Natural Intellects Control Center
6. Upgrade & Downgrade Mechanics

7. Feature Enforcement System
 8. Billing Integration Model
 9. Technical Implementation Model
 10. Security & Abuse Prevention
 11. Operational Workflows
 12. Scalability Strategy
 13. Future Expansion Model
-

1. PRODUCT STRATEGY OVERVIEW

The Natural Intellects system shall evolve into a **centralized SaaS platform** where:

- One codebase powers all schools
 - Features are enabled per school
 - Pricing determines access level
 - Administration is controlled centrally
-

Key Principle

Build once, enable many.

Instead of:

- Rebuilding per school
- Custom coding per client

The system shall:

- Deploy once
 - Configure per school
 - Control via feature flags
-

2. PLATFORM VS SCHOOL SYSTEMS

Old Model (Not Recommended Long-Term)

```text id="old1" School A → Full System Copy

School B → Full System Copy

School C → Full System Copy

Problems:

- Hard to maintain
- Expensive updates
- Inconsistent versions

---

## New Model (Recommended)

```text id="new1"

Natural Intellectuals Platform

↓

School A

School B

School C

All schools share:

- Same codebase
- Same updates
- Same infrastructure logic

Only configuration differs.

3. FEATURE FLAG ARCHITECTURE

Concept

Each feature is controlled by a toggle.

Example:

```text id="ff1" FEATURE\_LIBRARY = true FEATURE\_SMS = false FEATURE\_HOSTEL = true

```

Feature Table Structure

```sql id="ff2"
SchoolFeatures

id
schoolId
featureKey
isEnabled
planLevel
updatedAt
```

Runtime Behavior

System checks:

```
``typescript id="ff3" if (featureEnabled("LIBRARY")) { showLibraryModule() } else
{ showUpgradePrompt() }
```

```
---

## UI Behavior

Two states:

### Enabled

- Feature visible
- Full access

### Disabled

- Locked icon
- Upgrade prompt

---

# 4. SUBSCRIPTION PLAN SYSTEM

---

## Plan Structure

---
```

```
### Starter Plan

```text id="plan1"
Students
Teachers
Attendance
Fees
Academics
Website
Announcements
```

---

## Professional Plan

Adds:

```
```text id="plan2" Library SMS Notifications Parent Portal Advanced Reports
```

```
---

### Enterprise Plan

Adds:

```text id="plan3"
Inventory System
Hostel Management
Advanced Analytics
API Access
White-labeling
```

---

## Plan Mapping

Each school is assigned:

```
```text id="planmap" schoolId → planType → features
```

```
---

# 5. NATURAL INTELLECTS CONTROL CENTER

---

## Purpose

A master dashboard for managing:
```

- Schools
- Features
- Plans
- Billing
- Support
- System health

Main Dashboard Modules

Schools Overview

```text id="dash1"

List of all schools

Status: Active / Suspended / Trial

---

## Feature Management

```text id="dash2" Toggle features per school

Example:

```text id="dash2b"

St Mary's School

- ☒ Students
- ☒ Teachers
- ☒ Fees
- ☒ Attendance
- ☐ Library
- ☐ SMS

---

## Subscription Management

```text id="dash3" School → Plan → Payment Status

Support Center

```text id="dash4"

Tickets  
Issues  
Requests

## Analytics

```text id="dash5" Usage statistics per school Feature usage Revenue tracking

```
---  
  
# 6. UPGRADE & DOWNGRADE MECHANICS  
  
---  
  
## Upgrade Flow  
  
```text id="up1"  
School Requests Upgrade

↓

Payment Confirmed

↓

Feature Flags Updated

↓

Features Activated Instantly
```

## Downgrade Flow

```text id="up2" Subscription expires

↓

Non-plan features disabled

↓

Data retained but locked

```
## Important Rule

> No code changes required for upgrades.

Only database changes.

---

# 7. FEATURE ENFORCEMENT SYSTEM

---

## Backend Enforcement

```typescript id="enf1"
if (!school.hasFeature("SMS")) {
 throw new ForbiddenError("Upgrade required")
}
```

---

## Frontend Enforcement

```text id="enf2" Menu Item Hidden OR Locked

```
---

## Soft Lock Strategy

Instead of removing features:

- Show UI
- Block usage
- Display upgrade prompt

---

# 8. BILLING INTEGRATION MODEL

---

## Payment Triggers

- Monthly subscription
- Annual subscription
- Feature add-ons

---

## Billing Flow
```



```
```text id="bill1"
Payment Received
```

↓

```
Update Subscription Record
```

↓

```
Enable Features
```

↓

```
Log Transaction
```

---

## Billing Table

```
```sql id="bill2" Subscriptions
```

```
id schoolId planType status renewalDate paymentStatus
```

```
---
```

```
# 9. TECHNICAL IMPLEMENTATION MODEL
```

```
---
```

```
## Core Principle
```

```
Everything is driven by:
```

```
```text id="tech1"
```

```
Database Configuration
```

NOT hardcoded logic.

---

## Feature Check System

Central function:

```
```typescript id="tech2" getSchoolFeatures(schoolId)
```

Returns:

```
```json id="tech3"
{
 "students": true,
 "fees": true,
 "sms": false
}
```

---

## Caching Strategy

- Cache feature flags per school
- Refresh on update
- Avoid database overload

---

# 10. SECURITY & ABUSE PREVENTION

---

## Risks

- Feature bypass attempts
- Unauthorized upgrades
- Manual URL access

---

## Protection Layers

- Backend enforcement (primary)
- Frontend hiding (secondary)
- API validation (required)

---

## Rule

Frontend never controls security. Backend enforces everything.

---

# 11. OPERATIONAL WORKFLOWS

## Adding New Feature

```text id="op1" 1. Code feature once 2. Register feature key 3. Add to plans 4. Toggle per school

```
---

## Enabling Feature for School

```text id="op2"
1. Open Control Center
2. Select School
3. Toggle Feature ON
4. Save
5. Feature instantly active
```

---

## Removing Feature

```text id="op3" 1. Toggle OFF 2. Data retained 3. UI locked

```
---

# 12. SCALABILITY STRATEGY

---

## Horizontal Scaling

System supports:

- Multiple schools
- Multiple regions
- High concurrent usage

---

## Feature Expansion

New features can be:

- Added without breaking existing systems
- Rolled out gradually
- Tested per school

---

## Long-Term Vision
```

Natural Intellects becomes:

```
```text id="scale1"
Education Operating System (EduOS)
```

---

## 13. FUTURE EXPANSION MODEL

---

### Potential Additions

- AI learning assistant
  - Student performance prediction
  - National education reporting
  - Government integrations
  - Exam body integrations
- 

### Platform Evolution

From:

School Management System

To:

Full Education Infrastructure Platform

---

## STRATEGIC CONCLUSION

The Natural Intellects platform shall operate as:

- A centralized SaaS system
- A feature-flag controlled architecture
- A subscription-driven product
- A scalable education ecosystem

This model ensures:

- No repeated deployments per feature
- Instant upgrades per school
- Centralized control
- Flexible pricing models
- Long-term scalability

End of Document 11.